UNIVERSITY OF CENTRAL PUNJAB

Lahore, Pakistan

Faculty of Information Technology and Computer Science

Course: Cloud Computing

Assignment 1: Research Paper

Multiresource Accelerated Genetic Algorithm for Energy Efficient VM Placement in Cloud Data Centers

Submitted by:	Sumyia Butt Student ID: L1S24BSCS0067
Program and Section:	BSCS, Section F1
Submitted to:	Muhammad Zulkifl Hasan

Base paper:	Hormozi et al. (2022), Energy (Elsevier), doi: 10.1016/j.energy.2022.123884
Dataset:	Google Cluster Usage Traces
Submission date:	8 November 2026

Multiresource Accelerated Genetic Algorithm for Energy Efficient VM Placement in Cloud Data Centers

Sumyia Butt, Muhammad Zulkifl Hasan, and Muhammad Zunnurain Hussain

S. Butt is with the Faculty of Information Technology and Computer Science, University of Central Punjab, Lahore, Pakistan (ORCID: https://orcid.org/0009-0009-2595-0102).

M. Z. Hasan is with the Faculty of Information Technology and Computer Science, University of Central Punjab, Lahore, Pakistan (e-mail: zulkifl.hasan@ucp.edu.pk; ORCID: https://orcid.org/0000-0002-2733-5527). Corresponding author: Muhammad Zulkifl Hasan.

M. Z. Hussain is with the Department of Computer Science, Bahria University Lahore Campus, Lahore, Pakistan (e-mail: zunnurain.bulc@bahria.edu.pk; ORCID: https://orcid.org/0000-0002-6071-1029).

Abstract

Virtual machine placement strongly affects the energy use of cloud data centres. Genetic algorithms produce good placements but are costly, and recent accelerated variants simplify the fitness function to a residual resource term that looks at CPU only. This paper extends that idea to two resources. We study a multiresource accelerated genetic algorithm with three components: a multi seed initial population built from first fit decreasing variants, a fitness function that combines CPU and memory residual resources with a capacity penalty, and a targeted consolidation mutation. The method is evaluated on samples of the Google cluster usage traces at three scales (1350, 2242 and 5365 virtual machines). Placement uses requested resources, and SLA violations are assessed with measured usage over 5 minute windows, across five independent runs. Against a two resource first fit decreasing baseline, the proposed algorithm lowers energy by 3.1 to 3.7 percent and activates 3.4 to 11.8 fewer hosts, while SLA violations stay within run to run variation. Against CPU only methods it uses 2.4 to 3.7 percent more energy, and its SLA advantage is visible only at the small scale. An ablation study shows that the whole gain comes from selecting among the seeds with the multiresource fitness, while crossover, mutation and consolidation add nothing in any run. The energy saving is unchanged for power model shape constants of 2, 3 and 4.

Keywords: virtual machine placement, genetic algorithm, energy efficiency, multiresource fitness, SLA violation, cloud data centre

1 Introduction

Cloud data centres host a very large number of virtual machines (VMs) on physical machines (PMs), and the placement of VMs on PMs decides how many hosts stay powered on. Idle servers still draw a large share of their peak power, so packing VMs onto fewer hosts is one of the most direct ways to lower energy use [1]. Placement is an NP hard problem, and heuristics such as first fit decreasing (FFD) are fast but can leave room for improvement. Genetic algorithms (GAs) search the placement space globally, but each generation requires repeated fitness evaluation, which becomes expensive at large scale.

Hormozi et al. [1] addressed this cost by replacing the exponential power based fitness with a residual resource term obtained from a Taylor expansion, and reported about 8 percent lower energy than FFD and 66 percent lower GA execution time than a standard GA on Google trace samples. Their fitness considers CPU utilisation only, SLA violations are not evaluated, and placement is static. Other recent work considers several resources and SLA directly. Alourani et al. [2], for example, combine adaptive thresholds with a GA whose fitness weighs energy and SLA violations, but they evaluate with synthetic task types in a simulator at up to 300 VMs.

This paper studies whether the accelerated residual fitness of [1] remains useful when memory is treated as a second resource, and how such a method behaves when SLA violations are measured with real usage data. The contributions are as follows.

1. A multiresource residual fitness that combines CPU and memory residuals with a capacity penalty, which keeps the linear cost of the fitness of [1].

2. A multi seed initial population built from FFD orderings by CPU, memory, their maximum and their sum, combined with a targeted consolidation mutation.

3. A reproducible evaluation protocol on the Google cluster usage traces that places by requested resources and measures SLA violations on measured 5 minute usage, with full code released, together with an honest account of where the expected gains did not appear.

4. An ablation and sensitivity study that identifies which component produces the gain, and shows that the result holds for different shape constants of the power model and different fitness weights.

The rest of the paper is organised as follows. Section II reviews related work, Section III presents the method, Section IV describes the experimental setup and data, Section V reports results, ablation and sensitivity analyses, Section VI discusses them, Section VII covers threats to validity, and Section VIII concludes.

2 Related Work

GA based placement has been studied extensively. Hormozi et al. [1] formulate placement as a constrained optimisation problem and design a fitness function that sums squared host utilisations, derived from the standard power model by Taylor expansion. Their evaluation on 24 hour samples of the Google traces covers small, medium and large data centres, and they list static placement as a limitation. The article does not provide a link to code or processed data, which limits direct reproduction.

Alourani et al. [2] present a GA with adaptive thresholds that classify hosts as underloaded, normal or overloaded and add a middle threshold to use lightly loaded hosts better. Their fitness combines normalised energy and SLA violation, and their experiments use CloudSim with 100 hosts and 50 to 300 VMs. They report energy reductions between 13 and 15 percent over a utility based approach. Their host utilisation includes CPU, memory and storage through a weighted sum, so multiple resources are already common in this line of work, and our contribution should be read as a study of the accelerated residual fitness under two resources rather than as the first use of memory.

Other work targets related objectives. Li et al. [3] optimise multiple resources collaboratively for adaptive placement, Peake et al. [4] parallelise ant colony optimisation for placement, Muteeh et al. [5] balance multiple resources with ant colony optimisation, and Malik et al. [6] use queuing and thresholds for load balancing in workflow scheduling. On the autoscaling side, Pintye et al. [7] select relevant internal metrics to drive a machine learning autoscaler and report fewer QoS violations and fewer VMs, which points to a possible coupling between scaling and placement that we leave for future work. Table 1 summarises how the closest studies differ from the present one.

3 Proposed Methodology

3.1 System Model

A data centre has N homogeneous hosts with normalised capacity of 1 for CPU and 1 for memory. VM i requests c_i of CPU and m_i of memory, and a placement assigns each VM to one host. For host j the utilisations are u_j^c and u_j^m, the sums of the requests of the VMs assigned to it. A host is active if it holds at least one VM, and inactive hosts are switched off. Let $\mathcal{A}$ be the set of active hosts. Host power follows the model used in [1]:

$$P(u) = P_{\max} - \frac{P_{\max} - P_{\min}}{e^{\alpha u}} \tag{1}$$

where $P_{\min}$ and $P_{\max}$ are idle and peak power and α shapes the curve. The energy of a placement over a period T is

$$E = T \sum_{j \in \mathcal{A}} P\big(\min(u_j^c, 1)\big) \tag{2}$$

which keeps CPU as the only input of the power model, as in [1]. Memory therefore affects energy only through the number of hosts that must stay active.

3.2 Fitness Functions

Three fitness functions are compared. The standard GA minimises the energy sum of (2) directly. The base paper GA of [1] minimises the residual CPU term:

$$F_{\text{base}} = \sum_{j \in \mathcal{A}} \big(1 - (u_j^c)^2\big) \tag{3}$$

The proposed multiresource residual fitness adds memory and a penalty for capacity violations:

$$F_{\text{prop}} = \sum_{j \in \mathcal{A}} \Big[1 - \big(w(u_j^c)^2 + (1-w)(u_j^m)^2\big)\Big] + \lambda \sum_{j} \Big[\max(0, u_j^c - 1) + \max(0, u_j^m - 1)\Big] \tag{4}$$

with $w = 0.5$ and $\lambda = 1000$. Each evaluation needs one pass over the VMs and one pass over the hosts, so the cost stays linear in the number of VMs and active hosts, as for (3).

3.3 Genetic Algorithm

A chromosome stores the host index of every VM. The initial population contains FFD solutions obtained by sorting VMs by CPU, by memory, by the larger of the two, and by their sum. The solution with the best fitness becomes the elite, and the remaining individuals are copies in which 5 percent of the genes are moved to hosts already used in the seed. Parents are chosen by tournament selection with a tournament size of 16, offspring are produced by uniform crossover with rate 0.5, and each gene mutates with probability 0.015 by moving to a random used host.

In addition, with probability 0.3 a consolidation mutation selects one of the five least loaded active hosts and tries to move all of its VMs to other active hosts that have room for them in every considered resource. The move is applied only if the host can be emptied completely. The algorithm keeps the elite, stops after 50 generations without improvement, and allows at most 999 generations. The two CPU only GAs use a CPU only FFD seed and CPU only feasibility checks, so that every method uses the information its own fitness considers.

3.4 Algorithms and Complexity

Algorithm 1 gives the placement procedure and Algorithm 2 the consolidation mutation. Let n_v be the number

Table 1: Comparison of closely related studies.

Study	Method	Resources in fitness or load	SLA evaluated	Workload	Code or data
Hormozi et al. [1]	GA, residual fitness	CPU	No	Google trace sample	Not linked
Alourani et al. [2]	GA with adaptive thresholds	CPU, memory, storage (weighted)	Yes	CloudSim task types	Supplement
Pintye et al. [7]	ML autoscaler	Internal metrics (128)	QoS violations	Own cloud testbed	On request
This work	GA, multi seed, residual fitness	CPU and memory	Yes (measured usage)	Google trace sample	Open code

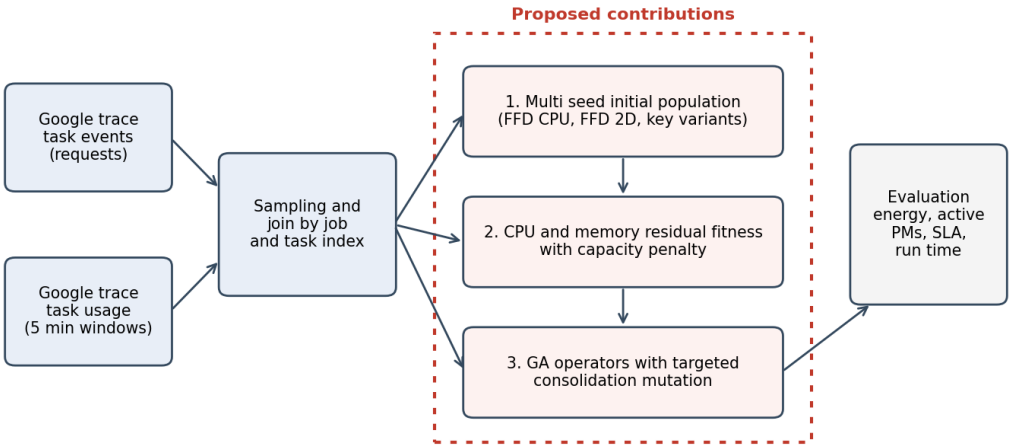


Figure 1: Overview of the proposed framework. The dotted box marks the three proposed contributions.

Algorithm 1. Multi seed multiresource placement

```
Input:  CPU and memory requests, N hosts
1 for key in {CPU, memory, max, sum}:
2   S_key = FFD(sorted by key, check CPU and memory)
3 elite = seed with lowest F_prop
4 P = elite + other seeds + perturbed copies
5 repeat
6   evaluate F_prop for every individual
7   select parents by tournament (size 16)
8   uniform crossover (0.5), mutation (0.015)
9   with probability 0.3:  consolidate(child)
10  keep elite
11 until 50 generations without improvement
12 return best placement
```

Algorithm 2. Consolidation mutation

```
1 pick one of the 5 least loaded active hosts h
2 for each VM v on h:
3   find an active host g with room for v
4   if none:  return x unchanged
5   move v to g (best fit by CPU)
6 return x with h emptied
```

of VMs and n_p the number of active hosts. One FFD seed costs $O(n_v \log n_v)$ for sorting plus the cost of finding a host with room, which is $O(n_v n_p)$ in the worst case. Evaluating F_{prop} costs $O(n_v + n_p)$. One generation with population P costs $O(P(n_v + n_p))$ for evaluation, and the consolidation mutation costs at most $O(n_v n_p)$ per application. Memory use is $O(P n_v)$. These costs match those of the CPU only residual fitness of [1], up to a constant factor for the second resource.

4 Experimental Setup

4.1 Data

We use the Google cluster usage traces released with [8]. Requests come from the task events table, where we keep submit events with positive CPU and memory requests and remove duplicate task identifiers. Usage comes from the task usage table, which reports mean CPU rate and memory usage over 5 minute windows. The two tables are joined on job and task index, and only tasks with at least 12 usage windows (one hour) are kept, so each retained task has a complete usage profile. Each run draws a different random sample of tasks. All values are normalised by the trace, so host capacity is 1 for both resources.

The scales follow Table 3 of [1]: small with 1350 VMs and 1000 hosts, medium with 2242 VMs and 2000 hosts, and large with 5365 VMs and 3000 hosts. We use one part file of each table, and the retained long running tasks are somewhat heavier than average, which we discuss as a limitation.

Table 2: Parameters of the experiments.

Parameter	Value
$P_{\min}$, $P_{\max}$ (W)	160, 220
Shape constant α	3
Population size	64
Tournament size	16
Crossover (uniform)	0.5
Mutation rate per gene	0.015
Consolidation probability	0.3
Stall limit, max generations	50, 999
Fitness weight w, penalty λ	0.5, 1000
Runs per scale	5 (seeds 0 to 4)

4.2 Methods and Metrics

Five methods are compared: FFD sorted by CPU (FFD CPU), FFD sorted by CPU with a memory check (FFD 2D), the standard GA, the base paper GA and the proposed GA. Metrics are the energy of (2) over 24 hours in kWh, the number of active hosts, the GA run time, and the SLA violation. We define the SLA violation as the percentage of pairs of an active host and a 5 minute window in which the measured CPU demand or memory demand of the assigned VMs exceeds the capacity of one. We also count hosts whose requested memory exceeds capacity, which shows what a CPU only method ignores.

4.3 Implementation and Parameters

All algorithms are implemented in Python 3.12.3 with NumPy 2.4.4, and the code is available as described in the data availability statement. Table 2 lists the parameters. The shape constant α is not given in [1], so we chose 3 to approximate the curve in their power model figure, and $P_{\min}$ and $P_{\max}$ are read from the same figure. The runs use seeds 0 to 4, and results are reported as mean and standard deviation over the five runs. The ablation and sensitivity experiments use all three scales with five runs each.

4.4 Sampled Workload

Fig. 2 and Table 3 describe the sampled tasks. Requests are small relative to a host, with a mean CPU request of 0.050 and a mean memory request of 0.038. CPU and memory requests are positively but not strongly correlated (correlation 0.47), so a task that is light in CPU can still be heavy in memory, which is the situation in which a CPU only method misplaces VMs. Measured usage is far below the request for most tasks (median ratio 0.08 for CPU and 0.25 for memory), yet about 13 percent of tasks use more than they requested, and these tasks produce the overload events counted in the SLA metric.

Table 3: Characteristics of the sampled tasks (mean over five small samples).

Quantity	CPU	Memory
Mean request per task (normalised)	0.050	0.038
Mean measured usage per task	0.017	0.021
Median of usage divided by request	0.08	0.25
Tasks with usage above request (%)	12.9	13.5
Largest request in a sample	0.38	0.31
Correlation of CPU and memory requests	0.47	

5 Results

5.1 Main Comparison

Table 4 and Figs. 3–5 report the main results, and Fig. 6 plots the energy and SLA of individual runs. FFD CPU, the standard GA and the base paper GA produce identical placements in every run, so their numbers coincide. With CPU requests only, FFD already packs close to the lower bound given by the total requested CPU (for the first small sample, 64.3 hosts of capacity 1 against 65 used by FFD), and no GA run improved on it.

Against FFD 2D, which respects memory as well, the proposed GA lowers energy by 3.7, 3.1 and 3.3 percent at the small, medium and large scales, and uses 3.4, 4.8 and 11.8 fewer hosts (Fig. 4). The per run differences in Table 5 are consistent across the five runs, with standard deviations well below the means.

The SLA results are less clear (Fig. 5). Relative to FFD 2D, the SLA difference is 0.45, −0.01 and 0.14 percentage points at the three scales, which is within the run to run standard deviation. Relative to FFD CPU, the proposed GA lowers the SLA violation by 1.73 points at the small scale, but the difference at the medium and large scales is not distinguishable from zero. Against CPU only methods, the proposed GA uses 2.4, 3.0 and 3.7 percent more energy, because it keeps more hosts active to respect memory.

5.2 Memory at Host Level

The memory check matters at the reservation level. On requested demand, the CPU only methods leave on average 11.6, 19.8 and 50.4 hosts at the three scales with more memory requested than the host provides, while FFD 2D and the proposed GA leave essentially none (at most 0.4 hosts on average). Fig. 7 shows the distribution for the first small sample. The CPU only placement fills hosts to a mean CPU utilisation of 99 percent but has 9 hosts above the memory capacity, with the worst host requesting 333 percent of it. The proposed placement keeps every host within capacity, with a mean CPU utilisation of 96 percent and a mean memory utilisation of 74 percent, compared with 71 percent for FFD 2D.

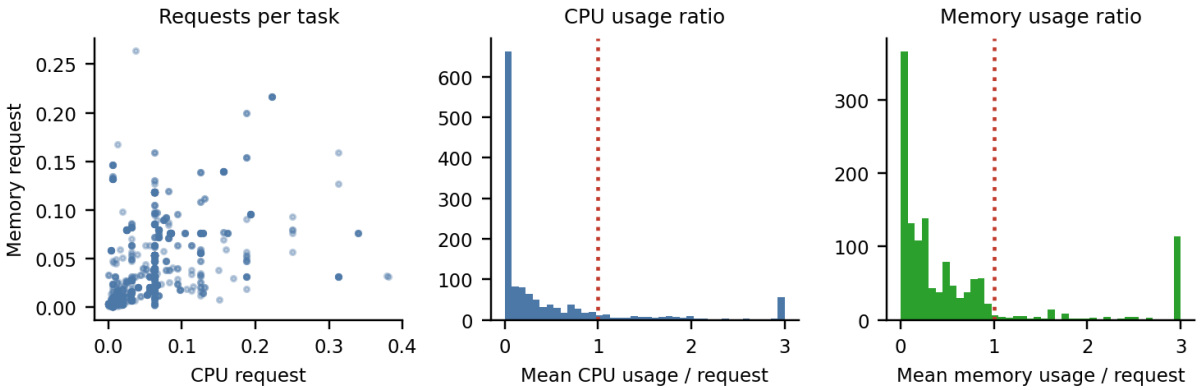


Figure 2: Sampled workload: CPU against memory requests, and the ratio of measured usage to request.

Table 4: Results on Google cluster usage trace samples (mean ± standard deviation over five runs). Run time of the FFD methods is below 0.3 s.

Scale	Method	Energy (kWh)	Active PMs	SLA (%)	Run time (s)
Small	FFD CPU	356.8±12.7	68.6±2.5	8.42±1.98	<0.3
	FFD 2D	379.5±10.5	73.8±1.9	6.23±1.53	<0.3
	Standard GA	356.8±12.7	68.6±2.5	8.42±1.98	0.69
	Base paper GA	356.8±12.7	68.6±2.5	8.42±1.98	0.68
	Proposed GA	365.5±10.7	70.4±2.1	6.68±1.51	0.71
Medium	FFD CPU	593.4±16.7	114.0±3.2	7.15±0.17	<0.3
	FFD 2D	630.7±15.3	122.4±3.0	7.21±0.61	<0.3
	Standard GA	593.4±16.7	114.0±3.2	7.15±0.17	0.78
	Base paper GA	593.4±16.7	114.0±3.2	7.15±0.17	0.85
	Proposed GA	611.0±13.3	117.6±2.5	7.20±1.12	0.84
Large	FFD CPU	1416.2±39.8	272.0±7.7	8.06±0.26	<0.3
	FFD 2D	1518.6±40.6	294.6±7.9	8.03±0.70	<0.3
	Standard GA	1416.2±39.8	272.0±7.7	8.06±0.26	1.42
	Base paper GA	1416.2±39.8	272.0±7.7	8.06±0.26	1.35
	Proposed GA	1468.4±37.5	282.8±7.2	8.17±1.03	1.48

5.3 Ablation

Table 6 removes one component at a time. Running the GA operators from the single FFD 2D seed returns the FFD 2D placement unchanged, so crossover, mutation and consolidation alone do not improve it. Selecting the best of the four seeds with the multiresource fitness, without any GA, already gives the result of the full proposed method, and adding the GA operators with or without consolidation changes nothing. The same pattern holds at all three scales. The whole gain over FFD 2D therefore comes from the seed ranking by F_{prop}, and the evolutionary part of the algorithm adds no improvement on these samples.

5.4 Sensitivity

Table 7 re evaluates the same placements with α equal to 2, 3 and 4. The saving of the proposed method against FFD 2D stays at about 3.7 percent at the small scale, about 3.1 percent at the medium scale and about 3.3 percent at the large scale for all three values, so the conclusion does not depend on our assumed shape constant. We also ran the proposed GA with fitness weights w of 0.3, 0.5 and 0.7. At the small and medium scales the energy, the number of hosts and the SLA violation were identical for all three weights, so the ranking of the four seeds does not change with w there. At the large scale, $w = 0.7$ selects a different seed in some runs and gives slightly lower values (1466.6 kWh, 282.4 hosts and 8.06 percent SLA against 1468.4 kWh, 282.8 hosts and 8.17 percent), a change of about 0.1 percent in energy.

5.5 Cost of Fitness Evaluation

Table 8 measures the time of a single fitness evaluation. The base paper fitness is faster than the standard energy sum by about 9 percent at the small scale and 12 percent at the medium scale, and the difference vanishes at the large scale. The proposed fitness costs more than the base fitness, by about 32 percent at the small scale and 31 percent at the large scale, because it reads memory as well. These differences are small in absolute terms, a few tens of microseconds, and much smaller than the 80 percent reduction of fitness time reported in [1] for their implementation. We attribute this to our vectorised NumPy

Table 5: Paired differences of the proposed GA against the baselines (mean ± standard deviation of the per run difference, five runs). 2D is FFD 2D and CPU is FFD CPU. Energy and PMs: proposed minus baseline, SLA in percentage points.

Scale	Energy vs 2D (%)	PMs vs 2D	SLA vs 2D (pts)	Energy vs CPU (%)	SLA vs CPU (pts)
Small	−3.7	−3.4±0.9	0.45±1.51	+2.4	−1.73±1.14
Medium	−3.1	−4.8±0.8	−0.01±0.85	+3.0	0.05±1.18
Large	−3.3	−11.8±1.9	0.14±0.83	+3.7	0.12±1.12

Table 6: Ablation study (mean over five runs). Energy in kWh, SLA in percent.

	Small			Medium			Large		
Variant	Energy	PMs	SLA	Energy	PMs	SLA	Energy	PMs	SLA
FFD 2D (single seed, no GA)	379.5	73.8	6.23	630.7	122.4	7.21	1518.6	294.6	8.03
Single seed + GA operators	379.5	73.8	6.23	630.7	122.4	7.21	1518.6	294.6	8.03
Multi seed selection only (no GA)	365.5	70.4	6.68	611.0	117.6	7.20	1468.4	282.8	8.17
Multi seed + GA, no consolidation	365.5	70.4	6.68	611.0	117.6	7.20	1468.4	282.8	8.17
Full proposed GA	365.5	70.4	6.68	611.0	117.6	7.20	1468.4	282.8	8.17

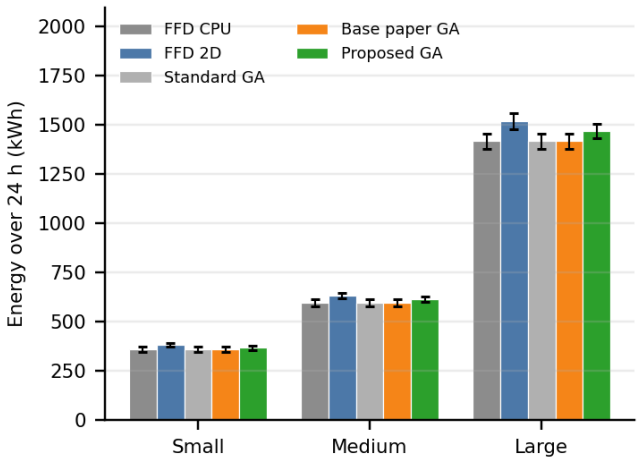


Figure 3: Energy over 24 hours (mean and standard deviation).

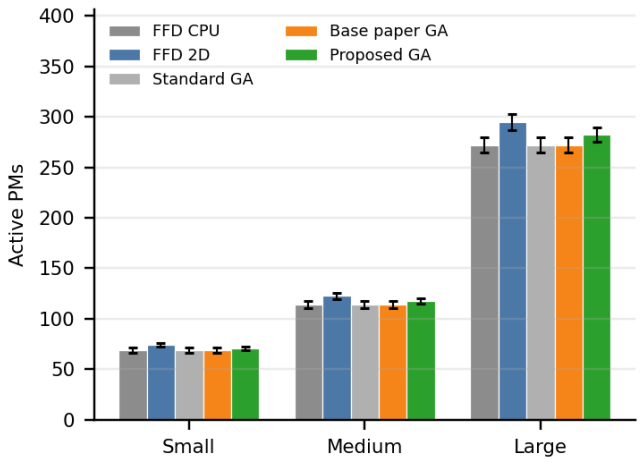


Figure 4: Number of active hosts (mean and standard deviation).

implementation, in which the exponential in the standard fitness is cheap.

5.6 Search Behaviour of the Genetic Algorithm

Table 10 reports the search statistics of the proposed GA. In all runs the algorithm stopped after the minimum of 51 generations, because no generation improved on the elite, and it evaluated 3328 fitness values, which is 52 evaluations of a population of 64. The run time therefore reflects only the cost of evaluating the initial population and one stall period, and it grows with the number of VMs from under one second to about one and a half seconds. This agrees with the ablation, and it shows that on these samples the seeds already sit at a point that the operators of Section III-C cannot improve.

6 Discussion

The central finding is a trade off. Treating memory as a second resource avoids memory overcommitment at reservation time and lowers energy compared with a plain two resource FFD, but it cannot match the energy of CPU only methods, since those methods pack on CPU alone. For a provider that must respect memory, the relevant baseline is FFD 2D, and against it the proposed method saves energy at all three scales. The SLA benefit that one might expect from memory awareness is small here and is clear only at the small scale, partly because measured usage in the trace is well below the requests for most tasks, so overload in measured usage is rare and driven by a minority of tasks whose usage exceeds their request.

Two results deserve a clear statement. First, the ablation shows that the evolutionary operators did not im-

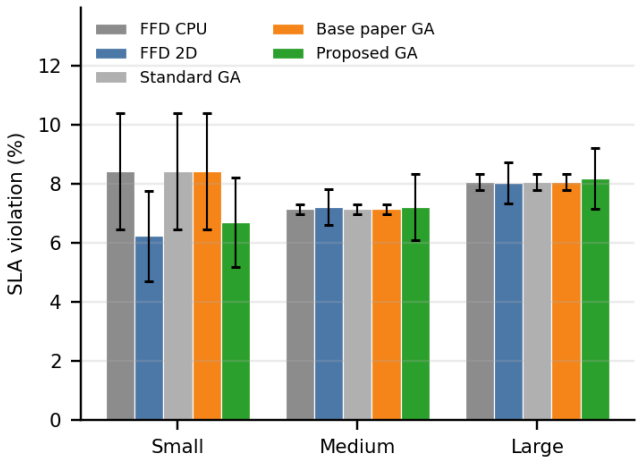


Figure 5: SLA violation (mean and standard deviation).

Table 7: Energy (kWh) for different shape constants α (placements computed with $\alpha = 3$, evaluated at each α). CPU is FFD CPU, 2D is FFD 2D, Prop. is the proposed GA.

Scale	α	CPU	2D	Prop.	vs 2D (%)
Small	2	348.3	370.1	356.5	−3.7
	3	356.8	379.5	365.5	−3.7
	4	360.0	383.4	369.0	−3.8
Medium	2	579.3	614.9	595.8	−3.1
	3	593.4	630.7	611.0	−3.1
	4	598.6	637.1	616.9	−3.2
Large	2	1382.6	1479.9	1431.5	−3.3
	3	1416.2	1518.6	1468.4	−3.3
	4	1428.6	1534.7	1482.8	−3.4

prove on the best seed in any run. The contribution of this work is therefore best described as a multiresource residual fitness used to rank several FFD seeds, and not as an improved genetic search. Second, we could not reproduce the execution time advantage reported in [1]. Our measurements show only a small difference between the standard and the residual fitness, and the proposed fitness is slightly more expensive. We therefore make no claim of acceleration beyond what the base paper reports on its own implementation.

A practical consequence is that a provider can obtain the reported saving with a simple procedure: run a few FFD variants and keep the one with the best residual fitness. A GA becomes useful only if its operators are strengthened so that they find placements the seeds do not contain, which we leave for future work.

6.1 Practical Implications

In the simulated setting, the saving of the proposed method against FFD 2D at the large scale is about 50 kWh over 24 hours (1518.6 against 1468.4 kWh), obtained with 11.8 fewer active hosts on average. A provider that already places by CPU and memory can therefore adopt the method with almost no change in its pipeline, because the extra work is a few additional FFD passes and one fitness evaluation per seed. The method also leaves the capacity constraints intact, so it can be used as a drop in replacement for the initial placement step of a scheduler. A provider that places by CPU alone would see lower energy in the model but would accept the memory overcommitment described in Section V-B.

Table 8: Time to evaluate one fitness value (μs, average of 300 evaluations).

Scale	Standard	Base paper	Proposed
Small	19.5	17.9	23.6
Medium	26.0	22.9	33.3
Large	52.7	52.2	68.5

7 Threats to Validity

Internal validity: the baselines were implemented by us, and the power model constant α is assumed, although Table 7 shows that the main conclusion is unaffected. The SLA metric is a proxy based on host overload in 5 minute windows and not a full simulator metric. *External validity:* the study uses one part file of each trace table and only tasks with at least one hour of usage, which biases the sample towards long running work, and results may differ for short tasks or other clusters. *Construct validity:* placement is static and does not model migrations, boot times or network effects, and the evaluation uses a custom simulator, not CloudSim. *Statistical validity:* five runs per scale give standard deviations that are large relative to some SLA differences, which is why we do not claim an SLA benefit beyond the small scale.

8 Conclusion

We examined a multiresource version of the accelerated residual fitness for GA based VM placement and evaluated it on Google cluster trace samples with SLA measured from real usage. The proposed method lowers energy by about 3 to 4 percent and uses fewer hosts than a two resource FFD baseline at three scales, a result that holds for different shape constants of the power model, while its SLA behaviour is similar to the baselines within run to run variation. Compared with CPU only methods it consumes more energy and shows an SLA benefit only at the small scale, but it avoids the memory overload that CPU only placements create. The ablation shows that the gain comes from ranking FFD seeds with the multiresource fitness, and that the evolutionary operators added no improvement. Future work should strengthen the operators, for example with swap based moves and repair, extend the study to dynamic placement with migrations, couple

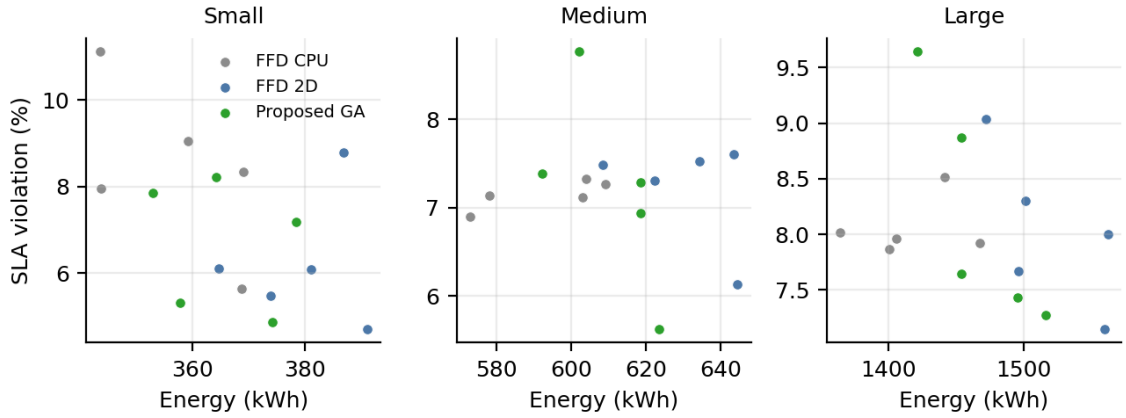


Figure 6: Energy and SLA violation of individual runs for FFD CPU, FFD 2D and the proposed GA.

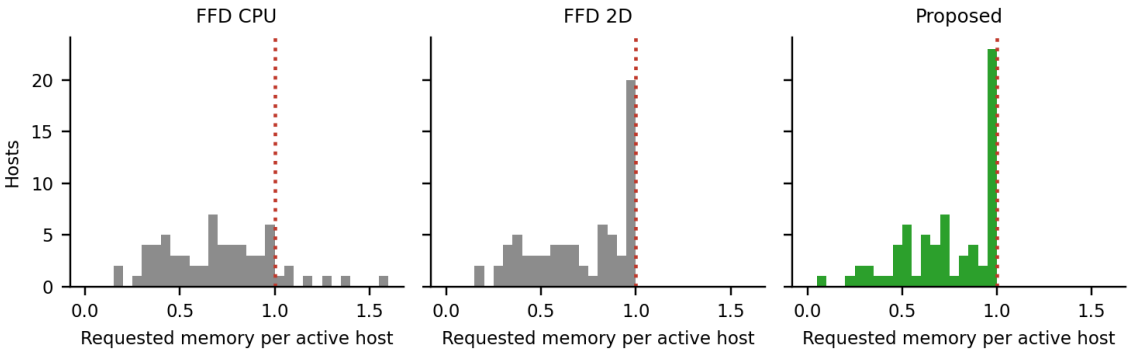


Figure 7: Requested memory per active host in the first small sample. The dotted line marks the host capacity.

placement with autoscaling as suggested by [7], and validate the results on more trace segments and in CloudSim.

Data Availability

The code, result files and analysis scripts are available at GitHub: `https://github.com/sumyiabutt/vm-placement-genetic-algorithm`. The Google cluster usage traces are publicly available from Google [8].

Acknowledgment

This research received no specific funding.

Declaration of AI-Assisted Technologies

During the preparation of this manuscript, the authors used Claude (Anthropic) to assist with drafting and editing the manuscript text, writing and debugging the code, and preparing figures and tables. The authors reviewed and verified the AI-assisted outputs and take full responsibility for the accuracy and integrity of the final manuscript.

References

[1] E. Hormozi, S. Hu, Z. Ding, Y. C. Tian, Y. G. Wang, Z. G. Yu, and W. Zhang, "Energy efficient virtual machine placement in data centres via an accelerated genetic algorithm with improved fitness computation," *Energy*, vol. 252, Art. no. 123884, 2022, doi: 10.1016/j.energy.2022.123884.

[2] A. Alourani, A. Khalid, M. Tahir, and M. Sardaraz, "Energy efficient virtual machines placement in cloud datacenters using genetic algorithm and adaptive thresholds," *PLoS ONE*, vol. 19, no. 1, Art. no. e0296399, 2024, doi: 10.1371/journal.pone.0296399.

[3] Z. Li, M. Pan, and L. Yu, "Multi resource collaborative optimization for adaptive virtual machine placement," *PeerJ Comput. Sci.*, vol. 8, Art. no. e852, 2022, doi: 10.7717/peerj-cs.852.

[4] J. Peake, M. Amos, N. Costen, G. Masala, and H. Lloyd, "PACO VMP: Parallel ant colony optimization for virtual machine placement," *Future Gener.*

Table 9: Per run results for FFD CPU (A), FFD 2D (B) and the proposed GA (C). Energy in kWh, SLA in percent.

		Energy (kWh)			Active PMs			SLA (%)		
Scale	Run	A	B	C	A	B	C	A	B	C
Small	1	343.4	364.6	352.9	66	71	68	11.11	6.10	7.84
	2	368.7	391.2	374.3	71	76	72	5.63	4.71	4.86
	3	343.7	374.0	357.8	66	73	69	7.95	5.48	5.31
	4	359.1	381.0	364.2	69	74	70	9.06	6.08	8.21
	5	369.0	386.9	378.4	71	75	73	8.33	8.78	7.19
Medium	1	572.9	608.4	592.2	110	118	114	6.89	7.49	7.38
	2	609.1	644.6	623.6	117	125	120	7.26	6.13	5.62
	3	578.1	622.4	602.0	111	121	116	7.13	7.30	8.76
	4	604.0	643.6	618.6	116	125	119	7.33	7.60	7.28
	5	603.0	634.4	618.5	116	123	119	7.11	7.52	6.93
Large	1	1400.7	1501.3	1453.9	269	291	280	7.87	8.30	7.65
	2	1467.9	1562.6	1516.5	282	303	292	7.92	8.00	7.28
	3	1364.5	1472.5	1421.8	262	286	274	8.02	9.03	9.64
	4	1441.9	1560.3	1495.5	277	303	288	8.51	7.15	7.44
	5	1406.1	1496.5	1454.3	270	290	280	7.96	7.67	8.87

Table 10: Search statistics of the proposed GA (mean over five runs).

Scale	Generations	Fitness evaluations	Run time (s)
Small	51	3328	0.71
Medium	51	3328	0.84
Large	51	3328	1.48

Comput. Syst., vol. 129, pp. 174–186, 2022, doi: 10.1016/j.future.2021.11.019.

[5] A. Muteeh, M. Sardaraz, and M. Tahir, "MrLBA: Multi resource load balancing algorithm for cloud computing using ant colony optimization," *Cluster Comput.*, vol. 24, no. 4, pp. 3135–3145, 2021, doi: 10.1007/s10586-021-03322-3.

[6] N. Malik, M. Sardaraz, M. Tahir, B. Shah, G. Ali, and F. Moreira, "Energy efficient load balancing algorithm for workflow scheduling in cloud data centers using queuing and thresholds," *Appl. Sci.*, vol. 11, no. 13, Art. no. 5849, 2021, doi: 10.3390/app11135849.

[7] I. Pintye, J. Kovács, and R. Lovas, "Enhancing machine learning based autoscaling for cloud resource orchestration," *J. Grid Comput.*, vol. 22, Art. no. 68, 2024, doi: 10.1007/s10723-024-09783-1.

[8] C. Reiss, J. Wilkes, and J. L. Hellerstein, "Google cluster usage traces: Format and schema," Google Inc., White Paper, 2011.